Exploiting XSS to Bypass CSRF Defenses

1. Overview

Field	Details
Vulnerability	Cross-Site Scripting (XSS)
Related Vulnerability	Cross-Site Request Forgery (CSRF)
Category	Client-Side Security
Attack Objective	CSRF Defense Bypass
Severity	High, depending on application functionality
Lab Platform	PortSwigger Web Security Academy
Vulnerability Type	XSS-Based CSRF Bypass

2. Description

Cross-Site Request Forgery (CSRF) defenses are designed to prevent unauthorized requests initiated from external websites.

Applications commonly use CSRF tokens, SameSite cookies, and origin validation to protect sensitive operations.

However, an existing Cross-Site Scripting vulnerability can undermine these protections.

Because injected JavaScript executes within the vulnerable application's origin, it may be able to access information available to the application's own scripts, including CSRF tokens exposed in the page or DOM.

The injected script may then use the victim's authenticated browser session to perform requests that satisfy the application's CSRF validation requirements.

3. Root Cause

The underlying problem is that XSS allows attacker-controlled JavaScript to execute within a trusted application origin.

This can undermine CSRF defenses because:

- JavaScript may access CSRF tokens exposed in the DOM.
- Same-origin requests may automatically include applicable session cookies.
- Injected scripts can interact with application forms and APIs.
- CSRF protections generally do not protect against malicious JavaScript executing within the trusted origin.

CSRF defenses and XSS prevention address different security threats.

4. Proof of Concept (PoC)

Testing environment: Authorized PortSwigger Web Security Academy lab.

1. Open the assigned lab.
2. Identify the existing XSS vulnerability.
3. Locate an application function protected by a CSRF token.
4. Inspect how the application generates and validates its CSRF token.
5. Determine whether the token is accessible through the page or DOM.
6. In the authorized lab, investigate whether JavaScript executing in the application's origin can access the token.
7. Verify whether a controlled, harmless test request can satisfy the application's CSRF validation.
8. Confirm the behavior using browser developer tools and the network panel.

Example conceptual request:

```
None
fetch('/example-endpoint', {
    method: 'POST',
    credentials: 'same-origin',
    headers: {
        'Content-Type': 'application/x-www-form-urlencoded'
    },
    body: 'csrf=LAB_TOKEN&example=TEST'
});
```

This illustrates how a same-origin request can include a token and applicable session cookies. The endpoint, parameter names, and token must be determined from the authorized lab; this is not a verified lab payload.

5. Security Impact

Successful exploitation may allow an attacker to:

- Circumvent CSRF protections through same-origin script execution.
- Perform unauthorized actions using the victim's authenticated session.
- Modify account settings or application data.
- Trigger sensitive application operations.
- Compromise the integrity of user accounts and application functionality.

The impact depends on the privileges of the victim and the functionality exposed by the application.

6. Recommended Remediation

- Eliminate XSS vulnerabilities using context-aware output encoding.
- Avoid unsafe DOM manipulation and inline JavaScript.
- Use secure templating frameworks.
- Implement CSRF tokens for state-changing requests.
- Validate CSRF tokens on the server side.
- Apply appropriate SameSite cookie settings.
- Validate Origin and Referer headers where appropriate.
- Avoid exposing unnecessary sensitive information to client-side scripts.
- Implement a restrictive Content Security Policy (CSP).
- Apply defense-in-depth controls to sensitive operations.

Important: CSRF tokens alone cannot reliably defend against XSS executing in the same origin. Preventing XSS is essential.

7. Verification

After remediation:

1. Retest the original XSS vulnerability.
2. Verify that injected JavaScript cannot execute.
3. Confirm CSRF tokens are validated server-side.
4. Test state-changing requests without valid tokens.
5. Verify that cross-origin requests are rejected where appropriate.
6. Confirm legitimate application functionality remains operational.

8. References

- PortSwigger: Cross-Site Scripting

- PortSwigger: Cross-Site Request Forgery
- PortSwigger: Exploiting XSS
- OWASP: Cross-Site Request Forgery Prevention Cheat Sheet
- OWASP: Cross-Site Scripting Prevention Cheat Sheet

9. Disclaimer

This report is intended for educational purposes and authorized security testing only. All testing should be performed within approved environments.